Before DevOps:

- Development and Operations teams worked in **isolation**
- Testing/deployment happened *after* design-build, causing major delays
- Teams had mismatched timelines and no synchronization
- Human errors were introduced during manual code deployment

What DevOps does: Bridges the gap between the Development ("Dev") team and Operations ("Ops") team by having them collaborate continuously. Developers package code in **containers** (lightweight software environments) that operations can run reliably.

The 8 Steps of DevOps (continuous loop):

1. **Plan** — gather feedback from monitoring, plan improvements
2. **Code** — developers write code on any platform
3. **Build** — compile/build the program version
4. **Test** — *must be automated* (e.g., using Selenium)
5. **Release** — manage, plan, schedule the build before deployment
6. **Deploy** — get code files ready and execute on the server
7. **Operate** — application runs in real-world scenarios
8. **Monitor** — track performance and uptime, feed back into planning

   Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (back to Plan)

Advantages of DevOps:

- Reduced chance of product failure
- Improved flexibility and support
- Faster time to market
- Better team efficiency
- Clearer product vision within the team

Challenges of DevOps:

- Difficulties with integration
- Automated testing complexity
- Relatively high costs
- Toolset choice (too many options)
- Lack of skilled talent

Segment 2: Software Lifecycle

The SDLC has **four main phases:**

Phase	Key Question	Deliverable
Planning	Why build the system?	Project Plan
Analysis	Who/what/when/where?	System Proposal
Design	How will it work?	System Specification
Implementation	Build & deliver	New System + Maintenance Plan

Segment 5: Project Team Roles

There are five key roles in an IS project team:

- **Business Analyst** — focuses on business value, processes, and policies
- **Systems Analyst** — bridges technology and business; designs the information system
- **Infrastructure Analyst** — ensures the system fits within the technical infrastructure
- **Change Management Analyst** — manages organizational change and user training
- **Project Manager** — leads the team, manages the plan, assigns resources, and serves as the main point of contact

What is a Requirement? A requirement is simply a statement describing what a system must do or what qualities it must have. There are five types:

Type	Meaning
Business Requirements	What the business needs
User Requirements	What users need to do
Functional Requirements	What the software should do
Non-Functional Requirements	How the system should behave/perform
System Requirements	How the system should be built

Functional Requirements describe the actual features and processes the system must perform to support users. For example, in a car dealership system: "The system will allow managers to view current vehicle inventory" or "The system will prepare a sales contract."

Non-Functional Requirements describe the quality and behavioral characteristics of the system — not *what* it does, but *how well* it does it. They fall into four categories:

- **Operational** — what hardware/environment it runs on (e.g., runs on Android, integrates with existing systems)

- **Performance** — speed, capacity, uptime (e.g., response time under 2 seconds, available 24/7)
- **Security** — who can access what (e.g., only managers can see staff records)
- **Cultural & Political** — legal and organizational rules (e.g., comply with Data Protection Act)

Functional = *what* the software does → easy to test, related to specific features

Non-Functional = *how* the software does it → harder to test, related to the system as a whole

Failing a functional requirement degrades the system; failing a non-functional requirement can make the **entire system unusable**

analysis strategies to understand what to improve:

Problem Analysis — Ask users what problems exist and what solutions they suggest. Simple but tends to produce only small, incremental improvements.

Root Cause Analysis — Instead of asking for solutions, ask users to list problems and dig into the *underlying causes*. More powerful because it finds common causes shared by multiple problems.

Duration Analysis — Measure how long each step of a process takes vs. how long the whole process takes. A big gap means the process is fragmented and inefficient. Solutions include:

- *Process Integration* — fewer people with broader responsibilities
- *Parallelization* — running steps simultaneously instead of one at a time

Activity-Based Costing — Calculate the cost of each step in a process (direct + indirect), then focus improvement efforts on the most expensive steps.

Benchmarking — Study how other organizations handle the same business process to find better ways of doing things. Informally, this can mean experiencing a competitor's process as a customer.

Technology Analysis — Analysts and managers brainstorm which emerging technologies could be applied to improve the business.

Activity Elimination — Ask "what would happen if we just removed this step entirely?" This challenges assumptions and can reveal unnecessary activities.

Comparing Techniques: When choosing which strategy to use, consider four factors — potential business value, project cost, breadth of analysis, and risk.

System Request

A formal document prepared by the **project sponsor** that includes:

- **Business Need** – why the system is needed
- **Business Requirements** – what the system should do
- **Business Value** – expected benefits (tangible or intangible)
- **Special Issues/Constraints** – deadlines, security needs, etc.

3. Feasibility Analysis

Assesses whether the project should proceed across **three dimensions:**

Type	Key Question	Focus
Technical	*Can* we build it?	Technology, team skills, project size, compatibility
Economic	*Should* we build it?	Cost-benefit analysis, ROI, BEP, NPV
Organizational	*Will they use* it?	Strategic alignment, stakeholder acceptance

Functional vs Nonfunctional — Key Differences

Functional (WHAT it does)	Nonfunctional (HOW it behaves)
Describes services or functions	Describes properties and constraints
Related to individual features	Related to the whole system
Easy to test	Difficult to test
Failure degrades the system	Failure may make system unusable
e.g. "System allows managers to view inventory"	e.g. "System runs on Android mobile devices"

💡 **Memory trick:** Functional = Features. Nonfunctional = Nature (speed, security, culture).

5 Requirement Gathering Techniques

1
Interviews
One-on-one conversations with stakeholders to understand needs and problems.

2
JAD Sessions (Joint Application Design)
Group workshops where users, managers, and analysts work together to define requirements.

3
Questionnaires
Written surveys sent to many stakeholders — good when people are spread out.

4
Document Analysis
Reviewing existing forms, reports, and manuals to understand the current system.

5
Observation
Watching users actually do their work — see what really happens, not just what they describe.

Requirements Analysis Strategies

These are techniques to understand and improve the current business before designing the new system:

Problem Analysis — Ask users what problems they have and what solutions they suggest. Gives small, incremental improvements. Rarely high business value.
Root Cause Analysis — Ask users for a prioritized list of problems and their root causes. Analysts investigate to find common causes and high-priority solutions.
Duration Analysis — Measure time for each step vs total process time. A big difference = fragmented process. Solutions: process integration or parallelization.
Activity-Based Costing — Calculate the cost of each step (direct + indirect). Focus improvement on the most expensive steps.
Benchmarking — Study how other organizations do the same process. Informal benchmarking = interact with competitors as if you're a customer.
Technology Analysis — Analysts and managers list interesting technologies, then identify how each could benefit the business.
Activity Elimination — Ask: what if we removed this step entirely? Use "force-fit" thinking to test all possibilities.

Q3. [10]

You have been hired to design a course management system. The requirements for the system are as follows:

Two types of students will use this system: BSc and MSc. Students can log in to the system using a username and password. The system will check whether the credentials are valid and give an error message if they are incorrect. Students will be able to select the courses they want to complete next semester. The system will check whether the course has remaining seats and if the student has completed all pre-requisite courses to take the course. The system might suggest additional courses to students while they are taking a course. After a student has completed course selection, they may request advising, and an adviser will approve the request. Students can view their routine and grade sheets. BSc students can add project proposals, whereas MSc students can add thesis proposals, which the advisor will approve. Students can also make payments. Payments can be made via card or bank transfer; the system must verify payment.